

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

Pooja Rajshekhar Arabi(1BM23CS413)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Sep-2024 to Jan-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Machine Learning (23CS6PCMAL)” carried out by **Pooja Rajshekhar Arabi (1BM23CS413)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

Lab faculty Incharge Name Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
--	--

Index

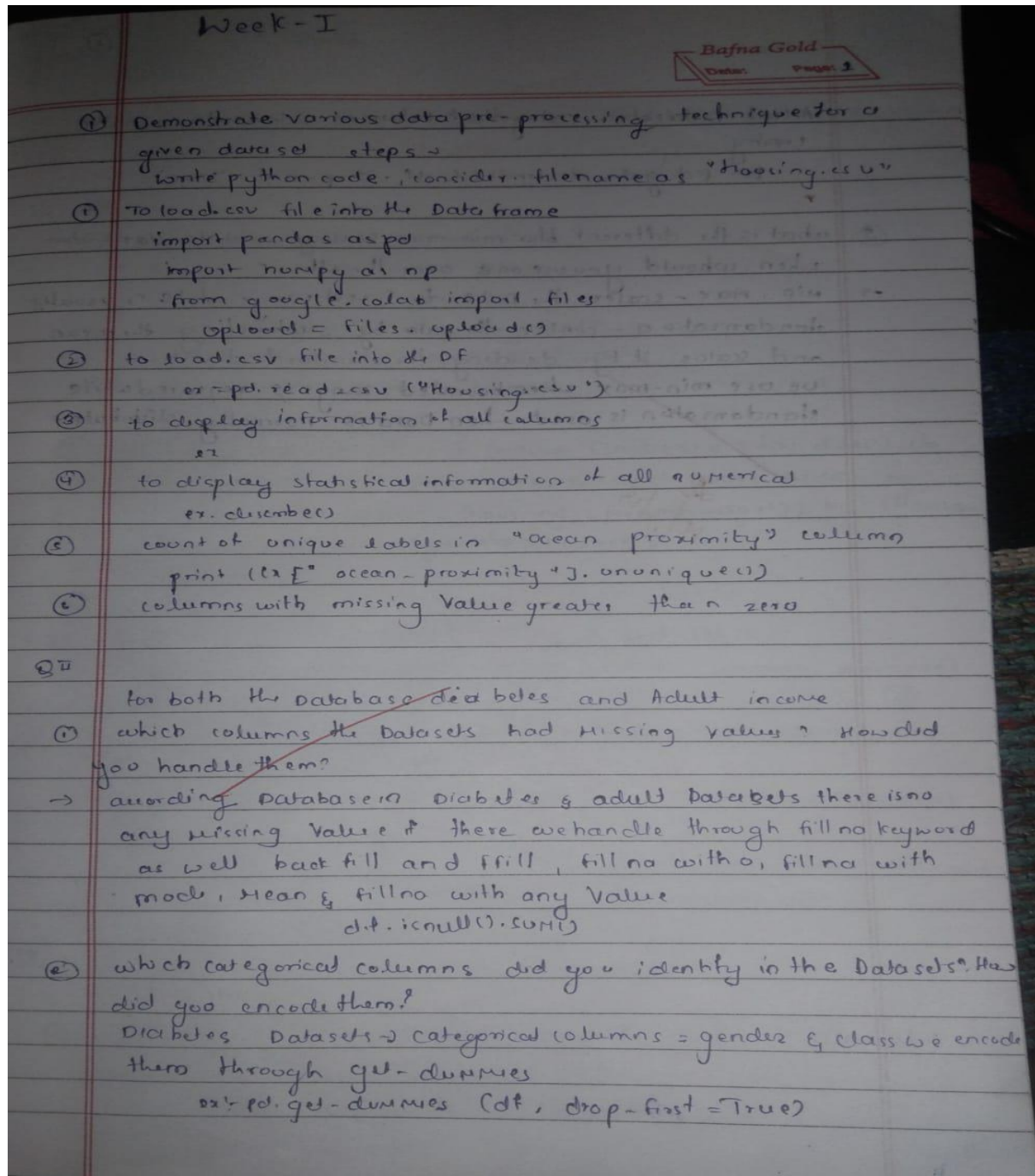
Sl. No.	Date	Experiment Title	Page No.
1	21-2-2025	Write a python program to import and export data using Pandas library functions	1
2	3-3-2025	Demonstrate various data pre-processing techniques for a given dataset	4
3	10-3-2025	Implement Linear and Multi-Linear Regression algorithm using appropriate dataset	9
4	17-3-2025	Build Logistic Regression Model for a given dataset	15
5	24-3-2025	Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.	20
6	7-4-2025	Build KNN Classification model for a given dataset.	25
7	21-4-2025	Build Support vector machine model for a given dataset	30
8	5-5-2025	Implement Random forest ensemble method on a given dataset.	35
9	5-5-2025	Implement Boosting ensemble method on a given dataset.	37
10	12-5-2025	Build k-Means algorithm to cluster a set of data stored in a .CSV file.	42
11	12-5-2025	Implement Dimensionality reduction using Principal Component Analysis (PCA) method.	47

Github Link: <https://github.com/pooja695/ML.git>

Program 1

1) Write a python program to import and export data using Pandas library functions

Screenshot:



Adult Datasets = categorical columns = work class, native country

Ex: `pd.get_dummies(df, drop_first=True)`

③ what is the different b/w min-max scaling and standardization
When would you use one over the other?

→ Min, Max - scales the data to fixed range while it's usually standardization - center the data by subtracting the mean and scales it by dividing the standard deviation.
We use min-max when bounded range is required while standardization is used when data is normally distributed.

10/3/25

Code:

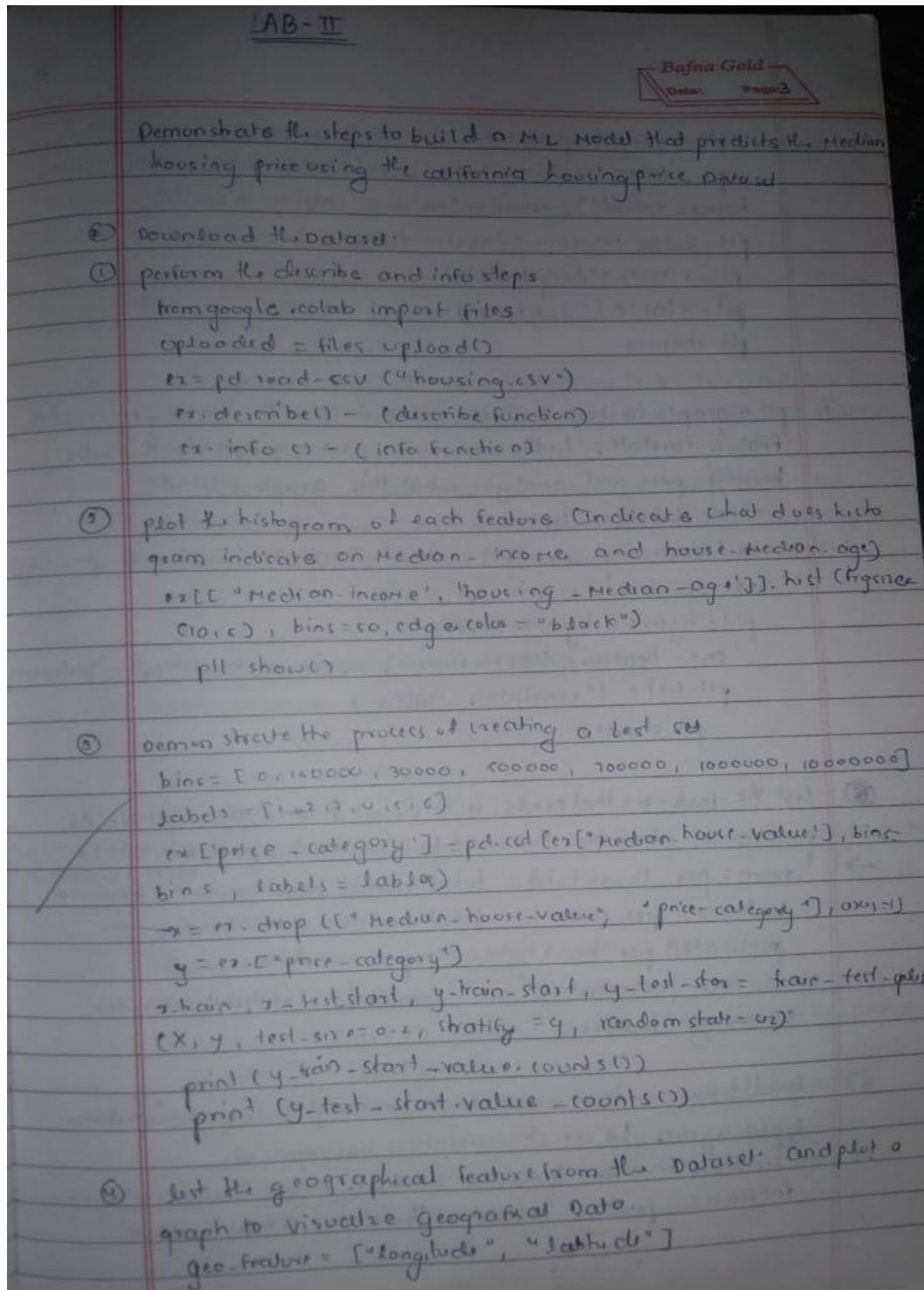
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder, StandardScaler, MinMaxScaler
from scipy import stats

df = pd.read_csv('/content/Dataset of Diabetes .csv')
print(df.head())
print("\nShape:", df.shape)
print("\nInfo:")
print(df.info())
print("\nSummary Statistics:")
print(df.describe())
missing_values = df.isnull().sum()
print("\nMissing Values:")
print(missing_values[missing_values > 0])
categorical_cols = df.select_dtypes(include=['object']).columns
print("\nCategorical Columns Identified:", categorical_cols)
if len(categorical_cols) > 0:
    df = pd.get_dummies(df, columns=categorical_cols, drop_first=True)
    print("\nDataFrame After One-Hot Encoding:")
    print(df.head())
else:
    print("\nNo Categorical Columns Found in the Dataset.")
numerical_cols = df.select_dtypes(include=['number']).columns
scaler = MinMaxScaler()
df_minmax = df.copy()
df_minmax[numerical_cols] = scaler.fit_transform(df[numerical_cols])
scaler = StandardScaler()
df_standard = df.copy()
df_standard[numerical_cols] = scaler.fit_transform(df[numerical_cols])
print("\nDataFrame after Min-Max Scaling:")
print(df_minmax.head())
print("\nDataFrame after Standardization:")
print(df_standard.head())

df1 = pd.read_csv('/content/adult.csv')
print(df1.head())
print("\nShape:", df1.shape)
print("\nInfo:")
print(df1.info())
missing_values = df1.isnull().sum()
print("\nMissing Values:")
print(missing_values[missing_values > 0])
print("\nSummary Statistics (numeric only):")
print(df1.describe())
```

Program 2

- 2) Demonstrate various data pre-processing techniques for a given dataset
Screenshot:




```
plt.figure(figsize=(10,6))
plt.scatter(ex["longitude"], ex["latitude"], c=ex["median
house-value"], cmap="rainbow", alpha=0.5, edgecolor="k")
plt.scatter_colorbar(label="median house value")
plt.xlabel("longitude")
plt.ylabel("latitude")
plt.show()
```

- ⑤ plot a graph to show features correlation with housing price. which feature correlates to the maximum. plot a graph for this with housing price and analyze what the graph indicates

```
le = LabelEncoder()
ex["ocean-proximity"] = le.fit_transform(ex["ocean-proximity"])
corr_matrix = ex.corr()
plt.figure(figsize=(10,8))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```

- ⑥ List the features that need to be cleaned and demonstrate the process of cleaning that could be combined to improve correlation. ^{to see if correlation improved}
- rooms per household = total rooms / household
- bedrooms per room = total bedrooms / total rooms
- population per household = population / household
- conclusion: correlation has remained constant even after combining features

- ⑦ List the feature that could be combined to improve correlation
- ↳ plot again to see if correlation has improved
- as median income increases, median house value also increases. positive correlation, there are a few outliers

① List the features that need to be cleaned and demonstrate the process of cleaning

→ Features that need to be cleaned include total-bedrooms, ocean-proximity, median-house-value. Cleaning of data is done by checking missing values, encoding categorical values, features scaling, outliers handling.

② Is there any categorical data that needs to be converted to numerical? If so, explain the method used to convert & look for some & show the o/p

→ The ocean-proximity column is categorical that can be converted to numerical data by using one-hot encoding to convert where the data is reshaped, transformed and then converted.

③ Discuss importance of feature scaling

→ Improve Model Performance

Faster convergence

enhanced interpretability

prevention of numerical issues

common scaling techniques are - min-max scaling, standardization, robust scaling.

④ Design a pipeline including Explain how it works
numerical pipeline is done by imputation (fills missing value with median value), custom transformations (adding features), scaling
categorical pipelining is done by one-hot encoding.

Code:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

# Load data
df = pd.read_csv('housing.csv')

# Histogram plots
sns.histplot(df['median_income'], kde=True, color='green')
plt.show()

sns.histplot(df['housing_median_age'])
plt.show()

# Train-test split with stratification
df["income_cat"] = pd.cut(df["median_house_value"],
                          bins=[0, 100000, 200000, 300000, 400000, np.inf],
                          labels=[1, 2, 3, 4, 5])
X = df.drop("median_house_value", axis=1)
y = df["median_house_value"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    stratify=df["income_cat"],
                                                    random_state=42)

# Geographical scatter plot
train_set = X_train.copy()
train_set["median_house_value"] = y_train
train_set.plot(kind="scatter", x="longitude", y="latitude", alpha=0.4,
                 s=train_set["population"]/100, label="population",
                 figsize=(10, 7), c="median_house_value", cmap=plt.get_cmap("jet"),
                 colorbar=True)
plt.legend()
plt.show()

# Correlation matrix
numerical_columns = df.select_dtypes(include=['float64', 'int64'])
correlation_matrix = numerical_columns.corr()
print(correlation_matrix["median_house_value"].sort_values(ascending=False))

# Scatter plot
df.plot(kind="scatter", x="median_income", y="median_house_value", alpha=0.1)
plt.show()

# Feature engineering
df["income_households"] = df["median_income"] * df["households"]

# Recalculate correlation
numerical_columns = df.select_dtypes(include=['float64', 'int64'])
correlation_matrix = numerical_columns.corr()
```

```

print(correlation_matrix["median_house_value"].sort_values(ascending=False))

df.plot(kind="scatter", x="income_households", y="median_house_value", alpha=0.1)
plt.show()

# Handle missing values
df = df.dropna(subset=["total_bedrooms"])

# One-hot encode categorical column
df1 = pd.read_csv('housing.csv')
hc = df1[["ocean_proximity"]]
encoder = OneHotEncoder()
hc_encoded = encoder.fit_transform(hc).toarray()
hc_1hot_df = pd.DataFrame(hc_encoded, columns=encoder.get_feature_names_out(hc.columns))
print(hc_1hot_df.head())

# Custom transformer for new features
class CombinedAttributesAdder(BaseEstimator, TransformerMixin):
    def __init__(self, add_bedrooms_per_room=True):
        self.add_bedrooms_per_room = add_bedrooms_per_room
    def fit(self, X, y=None):
        return self
    def transform(self, X):
        rooms_per_household = X[:, 3] / X[:, 5]
        population_per_household = X[:, 4] / X[:, 5]
        if self.add_bedrooms_per_room:
            bedrooms_per_room = X[:, 2] / X[:, 3]
            return np.c_[X, rooms_per_household, population_per_household, bedrooms_per_room]
        else:
            return np.c_[X, rooms_per_household, population_per_household]

# Identify columns
num_attribs = df1.drop("ocean_proximity", axis=1).columns
cat_attribs = ["ocean_proximity"]

# Numeric pipeline
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy="median")),
    ('attribs_adder', CombinedAttributesAdder()),
    ('std_scaler', StandardScaler()),
])

# Full pipeline
full_pipeline = ColumnTransformer([
    ("num", num_pipeline, num_attribs),
    ("cat", OneHotEncoder(), cat_attribs),
])

# Prepare dataset
housing = df1.copy()
housing_prepared = full_pipeline.fit_transform(housing)
print("Shape of processed data:", housing_prepared.shape)

```

Program 3

3) Implement Linear and Multi-Linear Regression algorithm using appropriate dataset

Screenshot:

lab - III

Implement linear and multi-linear regression algorithm using appropriate dataset

Solving the following Linear regression problem using Matrix approach. Find linear regression of the data of weekly product sales

x_i (weeks)	y_i (sales in thousands)
1	2
2	4
3	5
4	9

$$B = (X^T X)^{-1} X^T Y \quad \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \\ 1 & x_4 \end{bmatrix} \begin{bmatrix} B_0 \\ B_1 \end{bmatrix} + \begin{bmatrix} e_1 \\ e_2 \\ e_3 \\ e_4 \end{bmatrix}$$

→ ① Setup a design matrix X and the response vector Y

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix} \quad Y = \begin{bmatrix} 2 \\ 4 \\ 5 \\ 9 \end{bmatrix}$$

② Compute the matrix $X^T X$ and vector $X^T Y$:

$$X^T X = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \quad X^T Y = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} 2 \\ 4 \\ 5 \\ 9 \end{bmatrix}$$
$$X^T X = \begin{bmatrix} 1+1+1+1 & 1+2+3+4 \\ 1+2+3+4 & 1+4+9+16 \end{bmatrix} \quad X^T Y = \begin{bmatrix} 2+4+5+9 \\ 2+8+15+36 \end{bmatrix}$$
$$= \begin{bmatrix} 4 & 10 \\ 10 & 30 \end{bmatrix} \quad X^T Y = \begin{bmatrix} 20 \\ 61 \end{bmatrix}$$
$$B = (X^T X)^{-1} X^T Y$$
$$(X^T X)^{-1} = \frac{1}{10-100} \begin{bmatrix} 30 & -10 \\ -10 & 4 \end{bmatrix}$$
$$(X^T X)^{-1} = \frac{1}{-90} \begin{bmatrix} 30 & -10 \\ -10 & 4 \end{bmatrix}$$

$$\beta = \frac{1}{20} \begin{bmatrix} 30 & -10 \\ -10 & 4 \end{bmatrix} \begin{bmatrix} 20 \\ 61 \end{bmatrix}$$

$$\beta = \begin{bmatrix} 600 - 610 \\ -200 + 244 \end{bmatrix}$$

$$\beta = \frac{1}{20} \begin{bmatrix} -10 \\ 44 \end{bmatrix}$$

$$\beta = \begin{bmatrix} -0.5 \\ 2.2 \end{bmatrix}$$

$$\beta_0 = -0.5$$

$$\beta_1 = 2.2$$

$$y = -0.5 + 2.2x$$

normal/ simple linear regression Model -

Diameter (x)
in inches Price in
Dollars - predict the price of 20 inch pizza using
the data given

8	10
10	13
12	16

x_i	y_i	$x_i \times x_i$	$x_i \times y_i$
8	10	$8 \times 8 = 64$	80
10	13	$10 \times 10 = 100$	130
12	16	$12 \times 12 = 144$	192
SUM = 20	SUM = 39	SUM = 208	SUM = 402
Avg = 6.6	Avg = 13	Avg = 53	Avg = 88.6
$\bar{x} = 6.6$	$\bar{y} = 13$	$\bar{x^2} = 53$	$\bar{xy} = 88.6$

$$a_1 = (\bar{xy}) - (\bar{x})(\bar{y})$$

$$(\sum x_i^2) \times (\sum y_i)^2$$

$$a_1 = (88.6) - (6.6)(13)$$

$$a_1 = (88.6) - (85.8)$$

$$(56) - (43.56)$$

$$24.04$$

x_i	y_i	$x_i \times x_i$	$x_i \times y_i$
8	10	64	80
10	13	100	130
12	16	144	192
SUM = 30	SUM = 39	SUM = 308	SUM = 402
Avg = 10	Avg = 13	Avg = 102.6	Avg = 134

$$a_1 = \frac{(\sum xy) - (\sum x)(\sum y)}{(\sum x^2) - (\sum x)^2}$$

$$= \frac{(134) - (10)(13)}{(102.6) - (10^2)}$$

$$= \frac{134 - 130}{2.6} = \frac{4}{2.6}$$

$$a_1 = 1.538$$

$$a_0 = (\bar{y}) - (a_1)(\bar{x})$$

$$a_0 = (13) - (1.538)(10)$$

$$a_0 = (13) - 15.38$$

$$a_0 = -2.38$$

$$a_0 = -2$$

$$y = -2 + 1.5x$$

predict the price of 20 inch pizza using the data given

$$y = -2 + (1.5)(20)$$

$$y = -2 + 30$$

$$\boxed{y = 28}$$

① Data preprocessing steps

- * missing values - handled by imputing or dropping missing data
- * scaling - Applied to feature with different scales to ensure uniformity
- * encoding - categorical variables were encoded to convert them to numerical form for the model

② regression line for canada-per-capita-income.csv

yes, the regression line was plotted. It showed an upward trend in per capita income over the years, indicating a positive linear relationship b/w the year and per capita income

③ predicted salary for training Dataset

for a candidate 12 years of experience, a 10 in test score and 10 in interview score the model predicted a salary of [insert predicted salary value] \$ 64,629.25

④ encoding - "state one hot" encoding was used. The rest of the features were scaled to ensure consistent ranges & prevent large features from dominating the model

R/17

Code:

```
import pandas as pd
import numpy as np
from sklearn import linear_model
import matplotlib.pyplot as plt

df = pd.read_csv('/content/housing_area_price.csv')
plt.xlabel('area')
plt.ylabel('price')
plt.scatter(df.area, df.price, color='red', marker='+')

new_df = df.drop('price', axis='columns')
price = df.price

reg = linear_model.LinearRegression()
reg.fit(new_df, price)
reg.predict([[3300]])
reg.coef_
reg.intercept_
3300 * 135.78767123 + 180616.43835616432
reg.predict([[5000]])

df = pd.read_csv('/content/homeprices_Multiple_LR.csv')
df.bedrooms = df.bedrooms.fillna(df.bedrooms.median())

reg = linear_model.LinearRegression()
reg.fit(df.drop('price', axis='columns'), df.price)
reg.coef_
reg.intercept_
reg.predict([[3000, 3, 40]])
112.06244194 * 3000 + 23388.88007794 * 3 + -3231.71790863 * 40 + 221323.00186540384

df1 = pd.read_csv('/content/canada_per_capita_income.csv')
X = df1.year.values.reshape(-1, 1)
y = df1['per capita income (US$)']
```

```
model = LinearRegression()
model.fit(X, y)
model.predict([[2020]])
```

```
df1 = pd.read_csv('/content/canada_per_capita_income.csv')
X = df1.year.values.reshape(-1, 1)
y = df1['per capita income (US$)']
```

```
model = LinearRegression()
model.fit(X, y)
```

```
plt.figure(figsize=(8, 6))
plt.scatter(X, y, color='blue')
plt.plot(X, model.predict(X), color='red')
plt.xlabel('Year')
plt.ylabel('Per Capita Income (US$)')
plt.title('Per Capita Income in Canada over Time')
plt.grid(True)
plt.show()
```

```
df = pd.read_csv('/content/salary.csv')
X = df.iloc[:, :-1].values
y = df.iloc[:, 1].values
```

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='mean')
X = imputer.fit_transform(X)
```

```
model = LinearRegression()
model.fit(X, y)
model.predict([[12]])
```

```
df = pd.read_csv('/content/hiring.csv')
df['experience'] = pd.to_numeric(df['experience'], errors='coerce')
```

```
imputer = SimpleImputer(strategy='mean')
df['experience'] = imputer.fit_transform(df[['experience']])
df['test_score(out of 10)'] = imputer.fit_transform(df[['test_score(out of 10)']])
```

```

X = df.drop('salary($)', axis='columns')
y = df['salary($)']

model = LinearRegression()
model.fit(X, y)
model.predict([[2, 9, 6]])
model.predict([[12, 10, 10]])

df = pd.read_csv('/content/1000_Companies.csv')
X = df.iloc[:, :-1].values
y = df.iloc[:, 4].values

from sklearn.preprocessing import LabelEncoder, OneHotEncoder
from sklearn.compose import ColumnTransformer
labelencoder = LabelEncoder()
X[:, 3] = labelencoder.fit_transform(X[:, 3])

ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [3])], remainder='passthrough')
X = ct.fit_transform(X)
X = X[:, 1:]

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

regressor = LinearRegression()
regressor.fit(X_train, y_train)
regressor.predict([[1, 0, 91694.48, 515841.3, 11931.24]])

```

Program 4

4) Build Logistic Regression Model for a given dataset
Screenshots:

LAB-04

Build logistic regression for a given dataset

To consider a binary classification problem where we want to predict whether a student will pass or fail based on their study hours. A logistic regression model has been trained, and the learned parameters are $a_0 = -5$ $a_1 = 0.8$

Write a logistic regression equation for this problem

$$P(\text{Pass}|x) = \frac{1}{1 + e^{-(a_0 + a_1 x)}}$$

$a_0 = -5$
 $a_1 = 0.8$

$$\frac{1}{1 + e^{-(-5 + 0.8x)}}$$

Determine the predicted class for this student based on a threshold of 0.5

Calculate the probability that a student who studies for 7 hours will pass.

$$\begin{aligned} P(\text{Pass}|7) &= \frac{1}{1 + e^{-(-5 + 0.8 \times 7)}} \\ &= \frac{1}{1 + e^{-(5 + 5.6)}} \\ &= \frac{1}{1 + e^{-0.6}} \\ &= \frac{1}{1 + 0.5488} = \frac{1}{1.5488} \\ P(\text{Pass}|7) &= 0.6466110024 \end{aligned}$$

Determine the predicted class for this student based on a threshold value of 0.5

$P(\text{Pass}|x) \geq 0.5 = \text{Pass}$
 $P(\text{Pass}|x) < 0.5 = \text{Fail}$
 $x = 7 \text{ hours}$ $P(\text{Pass}|7) = 0.646611$
 $0.646611 > 0.5$
 $= \text{Pass}$

Predicted class: pass

② consider $z = [2.1 \ 1.0]$ for 3 class. Apply softmax function to find the probability values of 3 classes

$$e^{z_1} = e^2 = 7.389$$

$$e^{z_2} = e^1 = 2.718$$

$$e^{z_3} = e^0 = 1$$

$$\sum_{j=1}^3 e^{z_j} = e^2 + e^1 + e^0 = 7.389 + 2.718 + 1 = 11.107$$

$$\text{softmax}(z_1) = \frac{e^{z_1}}{\sum_{j=1}^3 e^{z_j}} = \frac{e^2}{11.107} = 7.389$$

$$\text{softmax}(z_2) = \frac{e^{z_2}}{\sum_{j=1}^3 e^{z_j}} = \frac{e^1}{11.107} = \frac{2.718}{11.107} \approx 0.245$$

$$\text{softmax}(z_3) = \frac{e^{z_3}}{\sum_{j=1}^3 e^{z_j}} = \frac{e^0}{11.107} = \frac{1}{11.107} \approx 0.090$$

① for dataset file "HR-commas-sep.csv"

① which variable did you identify as having a direct and clear impact on employee retention? why?

→ The key variable impacting the employee retention are:
 satisfaction level - lower satisfaction increase attrition time
 spent in company - employees with 5+ years tend to leave
 salary - low salaries lead to higher turn over not profit
 % avg Monthly hours - very high and low values affect retention

② → The accuracy of the logistic regression model is 78.40% the accuracy is overall good. But the model still needs improvements the model captures key factors

Code:

```
import pandas as pd
from matplotlib import pyplot as plt
# %matplotlib inline
#"matplotlib inline" will make your plot outputs appear and be stored within the notebook.

df = pd.read_csv("insurance_data.csv")
df.head()

plt.scatter(df.age,df.bought_insurance,marker='+',color='red')

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test =
train_test_split(df[['age']],df.bought_insurance,train_size=0.9,random_state=10)
X_train.shape

X_test

from sklearn.linear_model import LogisticRegression
model = LogisticRegression()

model.fit(X_train, y_train)

X_test

y_test

y_predicted = model.predict(X_test)
y_predicted

model.score(X_test,y_test)

model.predict_proba(X_test)

y_predicted = model.predict([[60]])
y_predicted

#model.coef_ indicates value of m in y=m*x + b equation
model.coef_

#model.intercept_ indicates value of b in y=m*x + b equation
model.intercept_

#Lets defined sigmoid function now and do the math with hand
import math
def sigmoid(x):
    return 1 / (1 + math.exp(-x))

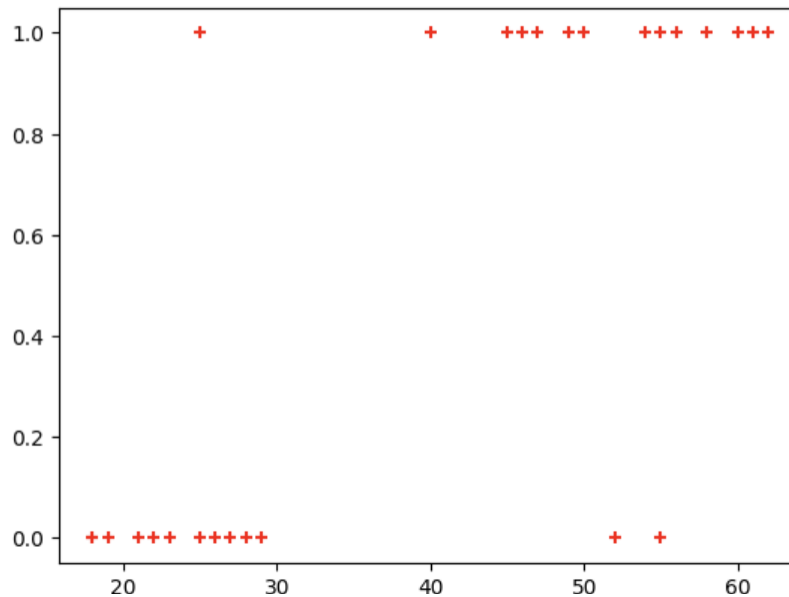
def prediction_function(age):
    z = 0.127 * age - 4.973 # 0.12740563 ~ 0.0127 and -4.97335111 ~ -4.97
    y = sigmoid(z)
    return y
```



```
age = 35
prediction_function(age)
```

""0.37 is less than 0.5 which means person with 35 will not buy the insurance""

'0.37 is less than 0.5 which means person with 35 will not buy the insurance'



Import necessary libraries

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn import metrics
import matplotlib.pyplot as plt
```

Load the Iris dataset

```
iris = pd.read_csv("iris.csv")
iris.head()
```

X=iris.drop('species',axis='columns')# Features (sepal length, sepal width, petal length, petal width)

y = iris.species # Target labels (0: Setosa, 1: Versicolor, 2: Virginica)

Split the dataset into 80% training and 20% testing

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Initialize the Multinomial Logistic Regression model

Use 'multinomial' for multi-class classification and 'lbfgs' solver

```
model = LogisticRegression(multi_class='multinomial')
```

Train the model on the training data

```
model.fit(X_train, y_train)
```

```

# Make predictions on the test data
y_pred = model.predict(X_test)

# Calculate the accuracy of the model on the test data
accuracy = accuracy_score(y_test, y_pred)

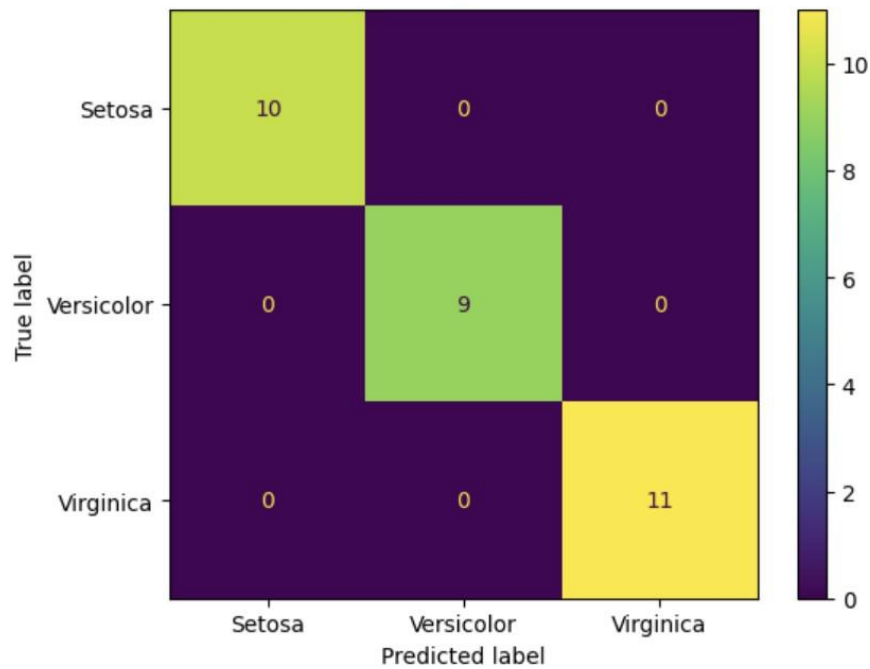
# Display the accuracy
print(f"Accuracy of the Multinomial Logistic Regression model on the test set: {accuracy:.2f}")

confusion_matrix = metrics.confusion_matrix(y_test, y_pred)

cm_display = metrics.ConfusionMatrixDisplay(confusion_matrix = confusion_matrix,
display_labels = ["Setosa", "Versicolor", "Virginica"])

cm_display.plot()
plt.show()
Accuracy of the Multinomial Logistic Regression model on the test set: 1.00

```



Program 5

5) Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.

Screenshots:

LAB-9

Decision Tree

Bafna Gold
 Date: Page: 13

consider the following dataset, calculate entropy and information gain w.r.t to target variable "classification". Identify whether the splitting node should be a_2 or a_3 attribute

Instance	a_2	a_3	classification
1	Hot	High	No
2	Hot	High	No
6	cool	High	No
7	Hot	High	No
8	Hot	Normal	Yes

Attribute a_2

$S_{a_2} = [1+4-]$ Entropy (S_{a_2}) = $-\frac{1}{5} \log_2 \frac{1}{5} - \frac{4}{5} \log_2 \frac{4}{5}$
 $= 0.7219$

$S_{Hot} = [1+3-]$ Entropy (S_{Hot}) = $-\frac{1}{4} \log_2 \frac{1}{4} - \frac{3}{4} \log_2 \frac{3}{4}$
 $= -\frac{1}{4} (-2.0000) - \frac{3}{4} (-0.415037)$
 $= 0.0000 + 0.311276$
 $= 0.311276$

$S_{Cool} = [0+1-]$ Entropy (S_{Cool}) = 0.0

~~Gain (S, a_2) = $\sum_{i=1}^n \frac{|S_i|}{|S|} \text{Entropy}(S_i)$~~

Gain (S, a_2) = Entropy (S) - $\frac{1}{5}$ Entropy (S_{Hot}) - $\frac{1}{5}$ Entropy (S_{Cool})
 $= 0.7219 - \frac{1}{5} (0.311276) - \frac{1}{5} (0.0)$
 $= 0.7219 - 0.062255$
 $= 0.659645$

Attribute a_3

$S_{a_3} = [1+4-]$ Entropy (S_{a_3}) = $-\frac{1}{5} \log_2 \frac{1}{5} - \frac{4}{5} \log_2 \frac{4}{5}$
 $= 0.7219$

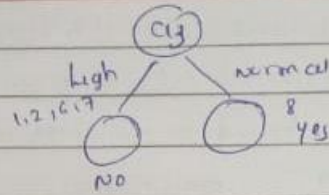
$S_{High} = [0+4-] = 0.0$
 $S_{Normal} = [1+0-] = 0.0$

Gain (S, a_3) = $0.7219 - 0.0 \times \frac{4}{5} - 0.0 \times \frac{1}{5} = 0.7219$

Gain (S, a_3) = 0.7219

Gain (S, a_2) = 0.659645

Gain (S, a_3) = 0.7219 - Max gain



① for 'ins.csv' dataset

→ a)

The accuracy score of the model was 1.00 (100%) meaning the model perfectly classified all test samples

b) The confusion matrix shows that the model made zero errors as all predictions matched the actual class

$\begin{bmatrix} 10 & 0 & 0 \\ 0 & 9 & 0 \\ 0 & 0 & 11 \end{bmatrix}$

$\begin{bmatrix} 0 & 9 & 0 \end{bmatrix}$

$\begin{bmatrix} 0 & 0 & 11 \end{bmatrix}$

c) No misclassification occurs since all off-diagonal values in the confusion matrix are zero. Every species were classified correctly without confusion

② for 'petrol-consumption.csv' dataset

→ a) The Regression Tree structure splits data to minimize MSE, with leaf node predicting the average petrol consumption

→ b) The most important features for predicting petrol consumption are petrol-tax and population-driver-license

→ c) The Regression Tree predicts continuous values (mean in leaves) while a classifier predicts categories (Majority class). The regression tree minimizes variance while the classifier minimizes impurity (Gini index or Entropy)

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

# Load the iris dataset (make sure iris.csv is in the working directory)
iris = pd.read_csv("iris.csv")
# Assuming the last column is the target (species) and the rest are features.
X = iris.iloc[:, :-1]
y = iris.iloc[:, -1]

# Split data into training and testing sets (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize and train the Decision Tree classifier
clf_iris = DecisionTreeClassifier(criterion='entropy', random_state=42)
clf_iris.fit(X_train, y_train)

# Make predictions and evaluate the model
y_pred_iris = clf_iris.predict(X_test)
accuracy_iris = accuracy_score(y_test, y_pred_iris)
conf_matrix_iris = confusion_matrix(y_test, y_pred_iris)

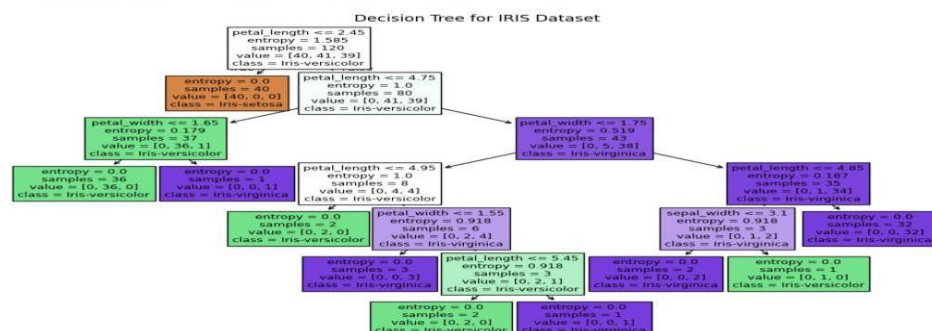
print("IRIS Dataset Decision Tree Classifier")
print("Accuracy:", accuracy_iris)
print("Confusion Matrix:\n", conf_matrix_iris)
print("Classification Report:\n", classification_report(y_test, y_pred_iris))

# Visualize the decision tree
plt.figure(figsize=(12, 8))
plot_tree(clf_iris, filled=True, feature_names=X.columns, class_names=clf_iris.classes_)
plt.title("Decision Tree for IRIS Dataset")
plt.show()
```

```
IRIS Dataset Decision Tree Classifier
Accuracy: 1.0
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
Classification Report:

```

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	10
Iris-versicolor	1.00	1.00	1.00	9
Iris-virginica	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30



```
import pandas as pd
```

```

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

# Load the drug dataset (make sure drug.csv is in the working directory)
drug = pd.read_csv("drug.csv")

# Since the target column is 'Drug', drop it from the features
X_drug = drug.drop('Drug', axis=1)
y_drug = drug['Drug']

# If there are categorical features, perform necessary encoding
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
# Encode features that are categorical
for col in X_drug.select_dtypes(include='object').columns:
    X_drug[col] = le.fit_transform(X_drug[col])
# Also encode the target variable if necessary
y_drug = le.fit_transform(y_drug)

# Split the data (80% training, 20% testing)
X_train_d, X_test_d, y_train_d, y_test_d = train_test_split(X_drug, y_drug, test_size=0.2,
random_state=42)

# Initialize and train the Decision Tree classifier using entropy criterion
clf_drug = DecisionTreeClassifier(criterion='entropy', random_state=42)
clf_drug.fit(X_train_d, y_train_d)

# Make predictions and evaluate the model
y_pred_drug = clf_drug.predict(X_test_d)
accuracy_drug = accuracy_score(y_test_d, y_pred_drug)
conf_matrix_drug = confusion_matrix(y_test_d, y_pred_drug)

print("Drug Dataset Decision Tree Classifier")
print("Accuracy:", accuracy_drug)
print("Confusion Matrix:\n", conf_matrix_drug)
print("Classification Report:\n", classification_report(y_test_d, y_pred_drug))

# Visualize the decision tree
plt.figure(figsize=(12, 8))
plot_tree(clf_drug, filled=True, feature_names=X_drug.columns,
class_names=[str(cls) for cls in clf_drug.classes_])
plt.title("Decision Tree for Drug Dataset")
plt.show()

```


Drug Dataset Decision Tree Classifier

Accuracy: 1.0

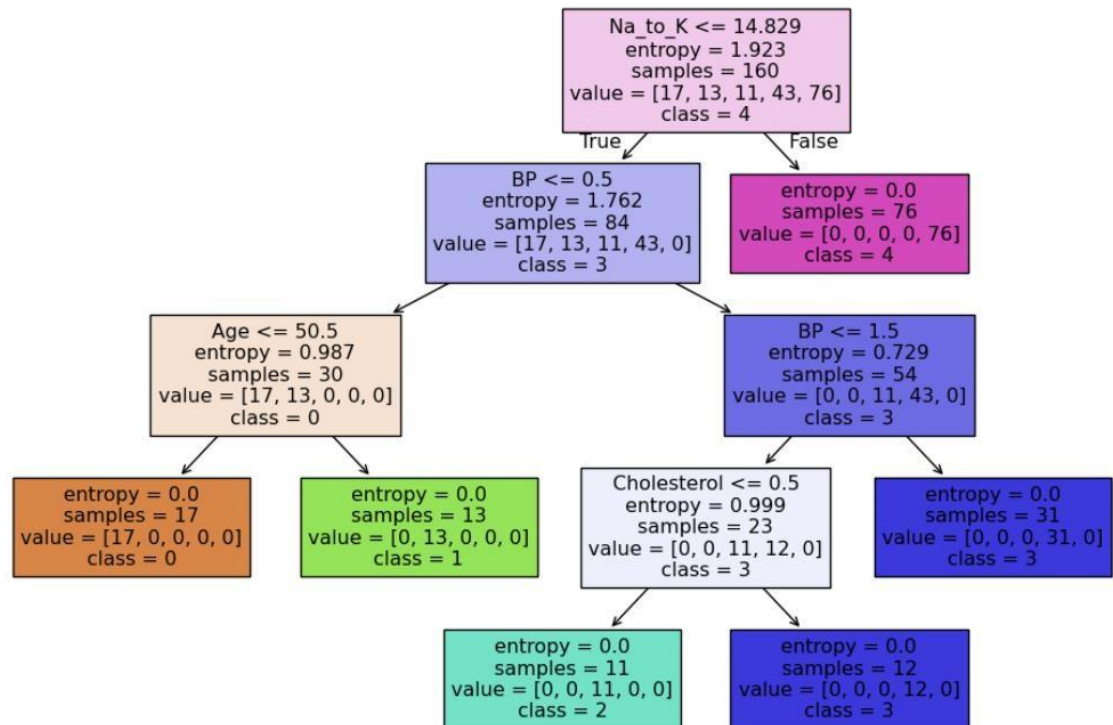
Confusion Matrix:

```
[[ 6  0  0  0  0]
 [ 0  3  0  0  0]
 [ 0  0  5  0  0]
 [ 0  0  0 11  0]
 [ 0  0  0  0 15]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	6
1	1.00	1.00	1.00	3
2	1.00	1.00	1.00	5
3	1.00	1.00	1.00	11
4	1.00	1.00	1.00	15
accuracy			1.00	40
macro avg	1.00	1.00	1.00	40
weighted avg	1.00	1.00	1.00	40

Decision Tree for Drug Dataset



Program 6

6) Build KNN Classification model for a given dataset.

Screenshots:

LAB - 6

Bafna Gold
Date: 7/4 Page: 15

① consider the following dataset, for $k=3$ and test data as above using knn classification Model & predict the target

person	Age	salary	Target
A	18	50	N
B	23	55	N
C	24	70	N
D	41	60	Y
E	43	70	Y
F	38	40	Y
X	35	100	?

distance $\rightarrow$

$$\sqrt{(35-18)^2 + (100-50)^2} = 52.81$$

$$\sqrt{(35-23)^2 + (100-55)^2} = 46.52$$

$$\sqrt{(35-24)^2 + (100-70)^2} = 31.95$$

$$\sqrt{(35-41)^2 + (100-60)^2} = 40.45$$

$$\sqrt{(35-43)^2 + (100-70)^2} = 31.05$$

$$\sqrt{(35-38)^2 + (100-40)^2} = 61.07$$

$k=1$ target = Y
 $k=2$ target = N
 $k=3$ target = Y
 target for person X = Y

② for iris dataset.

③ how to choose the k value. Demonstrate using accuracy rate & error rate.

$\rightarrow$ Best k value in iris dataset and error rate. The k values from 1 to 10 are tested for accuracy and error rate. The k value of 3 was chosen because of high accuracy and low error rate.

④ for diabetes datasets

⑤ what is the purpose of feature scaling. how to perform it.

$\rightarrow$ feature scaling in diabetes to ensure that all the features like glucose, age, insulin are on the same scale. Not normalized by large numerical values.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# For model building and evaluation
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
# ----- Part 1: IRIS Dataset ----- #
# Load the iris dataset (ensure iris.csv is in the same directory or provide correct path)
iris_df = pd.read_csv("iris.csv")

# Separate features and target
X_iris = iris_df.drop("species", axis=1)
y_iris = iris_df["species"]

# Split the data (80% training, 20% testing)
X_train_iris, X_test_iris, y_train_iris, y_test_iris = train_test_split(
    X_iris, y_iris, test_size=0.2, random_state=42
)

# Choose a value for k; here K=3 is used as an example.
knn_iris = KNeighborsClassifier(n_neighbors=3)

# Train the model on training data
knn_iris.fit(X_train_iris, y_train_iris)

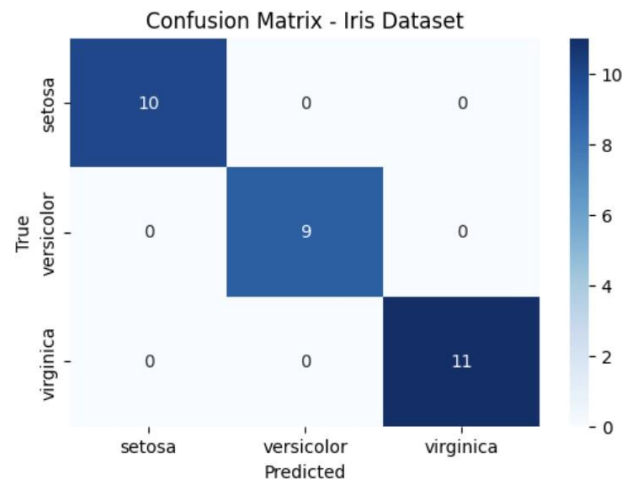
# Predict on test data
y_pred_iris = knn_iris.predict(X_test_iris)
# Calculate accuracy score
acc_iris = accuracy_score(y_test_iris, y_pred_iris)
print("IRIS Dataset Accuracy Score:", acc_iris)
# Compute confusion matrix and classification report
cm_iris = confusion_matrix(y_test_iris, y_pred_iris)
print("\nIRIS Dataset Confusion Matrix:\n", cm_iris)
```

```
cr_iris = classification_report(y_test_iris, y_pred_iris)
print("\nIRIS Dataset Classification Report:\n", cr_iris)
```

```
IRIS Dataset Classification Report:
              precision    recall  f1-score   support

   setosa         1.00        1.00        1.00         10
  versicolor      1.00        1.00        1.00          9
   virginica      1.00        1.00        1.00         11

 accuracy
macro avg         1.00        1.00        1.00         30
weighted avg      1.00        1.00        1.00         30
```



```
# ----- Part 2: Diabetes Dataset ----- #
# Load the diabetes dataset (ensure diabetes.csv is in the same directory or provide correct path)
diabetes_df = pd.read_csv("diabetes.csv")

# Separate features and target (Outcome column is assumed to be the target)
X_diabetes = diabetes_df.drop("Outcome", axis=1)
y_diabetes = diabetes_df["Outcome"]

# Perform feature scaling on the features
scaler = StandardScaler()
X_scaled_diabetes = scaler.fit_transform(X_diabetes)

# Split the scaled data (80% training, 20% testing)
X_train_diab, X_test_diab, y_train_diab, y_test_diab = train_test_split(
    X_scaled_diabetes, y_diabetes, test_size=0.2, random_state=42
)

# Choose a value for k; here K=5 is used as an example.
knn_diabetes = KNeighborsClassifier(n_neighbors=5)

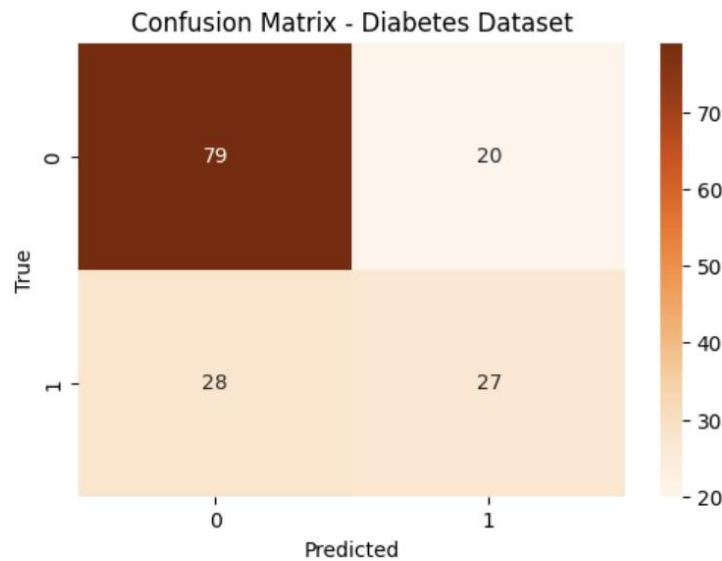
# Train the model on training data
knn_diabetes.fit(X_train_diab, y_train_diab)

# Predict on test data
y_pred_diab = knn_diabetes.predict(X_test_diab)
# Calculate accuracy score
acc_diab = accuracy_score(y_test_diab, y_pred_diab)
print("Diabetes Dataset Accuracy Score:", acc_diab)
# Compute confusion matrix and classification report
cm_diab = confusion_matrix(y_test_diab, y_pred_diab)
print("\nDiabetes Dataset Confusion Matrix:\n", cm_diab)
```

```
cr_diab = classification_report(y_test_diab, y_pred_diab)
print("\nDiabetes Dataset Classification Report:\n", cr_diab)
```

Diabetes Dataset Classification Report:

	precision	recall	f1-score	support
0	0.74	0.80	0.77	99
1	0.57	0.49	0.53	55
accuracy			0.69	154
macro avg	0.66	0.64	0.65	154
weighted avg	0.68	0.69	0.68	154



```
# ----- Load the Dataset ----- #
# Load heart.csv (make sure the file is in your working directory)
heart_df = pd.read_csv("heart.csv")

# Display the first few rows to check the data
heart_df.head()

# ----- Data Preparation ----- #
# Separate features and target
X_heart = heart_df.drop("target", axis=1)
y_heart = heart_df["target"]

# Perform feature scaling (important for distance-based algorithms like KNN)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_heart)

# Split data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y_heart, test_size=0.2, random_state=42)
# ----- Finding the Best k ----- #
# We will try a range of k values (neighbors) and select the one with maximum accuracy.
k_range = range(1, 21)
accuracy_scores = []

for k in k_range:
```

```

knn = KNeighborsClassifier(n_neighbors=k)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
acc = accuracy_score(y_test, y_pred)
accuracy_scores.append(acc)
print(f"k = {k} --> Accuracy: {acc:.4f}")

```

```

k = 1 --> Accuracy: 0.8525
k = 2 --> Accuracy: 0.8197
k = 3 --> Accuracy: 0.8689
k = 4 --> Accuracy: 0.8852
k = 5 --> Accuracy: 0.9180
k = 6 --> Accuracy: 0.9344
k = 7 --> Accuracy: 0.9180
k = 8 --> Accuracy: 0.8525
k = 9 --> Accuracy: 0.8852
k = 10 --> Accuracy: 0.8852
k = 11 --> Accuracy: 0.8852
k = 12 --> Accuracy: 0.8689
k = 13 --> Accuracy: 0.8852
k = 14 --> Accuracy: 0.8689
k = 15 --> Accuracy: 0.9016
k = 16 --> Accuracy: 0.8852
k = 17 --> Accuracy: 0.8852
k = 18 --> Accuracy: 0.9016
k = 19 --> Accuracy: 0.8852
k = 20 --> Accuracy: 0.8852

```

```

|: # Determine the best k value
   best_k = k_range[np.argmax(accuracy_scores)]
   print("\nBest k value:", best_k)

```

Best k value: 6

```

# ----- Train Final Model with Best k ----- #
best_knn = KNeighborsClassifier(n_neighbors=best_k)
best_knn.fit(X_train, y_train)
y_pred_best = best_knn.predict(X_test)

```

```

# Compute final accuracy, confusion matrix and classification report
final_accuracy = accuracy_score(y_test, y_pred_best)
cm = confusion_matrix(y_test, y_pred_best)
cr_text = classification_report(y_test, y_pred_best)
print("\nFinal Accuracy Score:", final_accuracy)
print("\nConfusion Matrix:\n", cm)
print("\nClassification Report:\n", cr_text)

```

Final Accuracy Score: 0.9344262295081968

Confusion Matrix:

```

[[28  1]
 [ 3 29]]

```

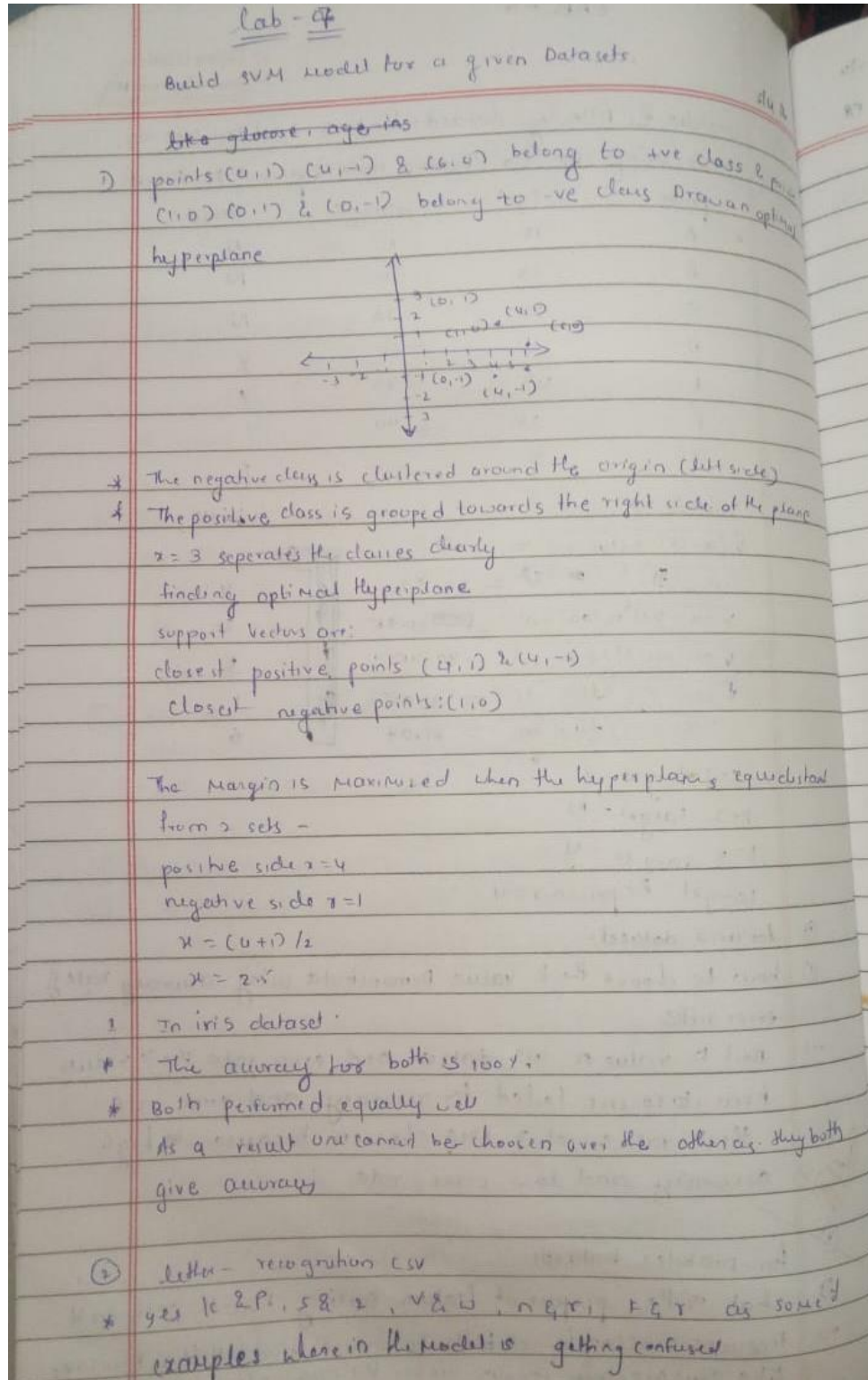
Classification Report:

	precision	recall	f1-score	support
0	0.90	0.97	0.93	29
1	0.97	0.91	0.94	32
accuracy			0.93	61
macro avg	0.93	0.94	0.93	61
weighted avg	0.94	0.93	0.93	61

Program 7

4) Build Support vector machine model for a given dataset

Screenshots:



- * area under the curve = $0.99 > 0.46$
- * The model is excellent at separating the classes across threshold in iris it is 100% accuracy for letter-recognition the accuracy is 93% (0.9305) it performed better on iris dataset than it did on the other i.e. the letter-recognition dataset

confusion Matrix

Linear Kernel	RBF Kernel
accuracy - 1.0	accuracy - 1.0
confusion Matrix	confusion Matrix
$\begin{bmatrix} 10 & 0 & 0 \\ 0 & 9 & 0 \\ 0 & 0 & 11 \end{bmatrix}$	$\begin{bmatrix} 10 & 0 & 0 \\ 0 & 9 & 0 \\ 0 & 0 & 11 \end{bmatrix}$

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC

# Data points
X = np.array([[4, 1], [4, -1], [6, 0], [1, 0], [0, 1], [0, -1]])
y = np.array([1, 1, 1, -1, -1, -1])

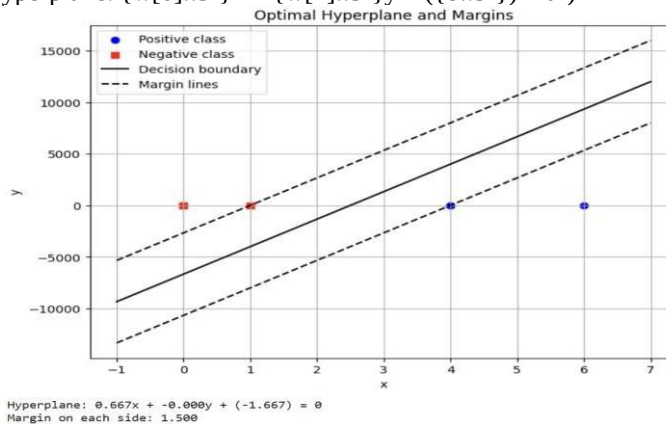
# Fit linear SVM with a very large C to approximate hard-margin
clf = SVC(kernel='linear', C=1e6)
clf.fit(X, y)

# Extract model parameters
w = clf.coef_[0]
b = clf.intercept_[0]

# Compute decision boundary and margins
xx = np.linspace(-1, 7, 500)
yy = -(w[0] * xx + b) / w[1]

# Margin offset: distance = 1/||w||
margin = 1 / np.linalg.norm(w)
yy_down = yy - np.sqrt(1 + (w[0] / w[1])**2) * margin
yy_up = yy + np.sqrt(1 + (w[0] / w[1])**2) * margin

# Plotting
plt.figure(figsize=(8, 6))
plt.scatter(X[y == 1, 0], X[y == 1, 1], c='blue', marker='o', label='Positive class')
plt.scatter(X[y == -1, 0], X[y == -1, 1], c='red', marker='s', label='Negative class')
plt.plot(xx, yy, 'k-', label='Decision boundary')
plt.plot(xx, yy_down, 'k--', label='Margin lines')
plt.plot(xx, yy_up, 'k--')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.title('Optimal Hyperplane and Margins')
plt.grid(True)
plt.show()
print(f"Hyperplane: {w[0]:.3f}x + {w[1]:.3f}y + ({b:.3f}) = 0")
```



```
print(f"Margin on each side: {margin:.3f}")
```

```
import pandas as pd
```

```

# Load both datasets
iris_df = pd.read_csv("/content/iris.csv")
# 1. IRIS DATASET - SVM with RBF and Linear Kernels
X_iris = iris_df.drop("species", axis=1)
y_iris = iris_df["species"]

# Encode labels
le_iris = LabelEncoder()
y_iris_encoded = le_iris.fit_transform(y_iris)

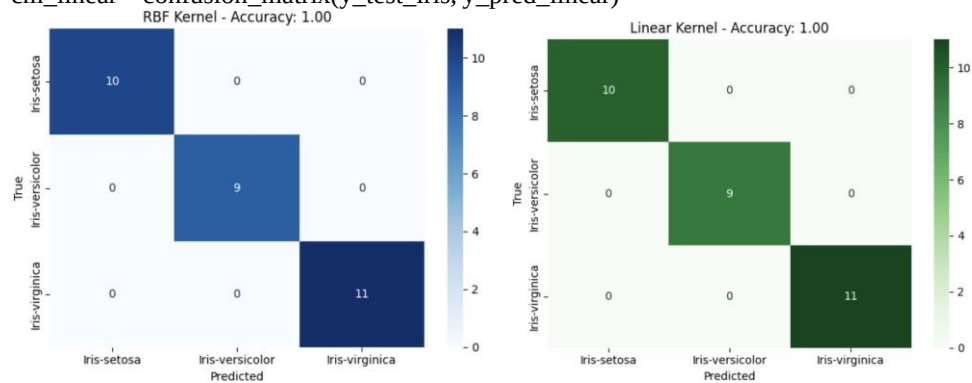
# Split dataset
X_train_iris, X_test_iris, y_train_iris, y_test_iris = train_test_split(X_iris, y_iris_encoded, test_size=0.2,
random_state=42)

# Train models
svm_rbf = SVC(kernel='rbf')
svm_linear = SVC(kernel='linear')

svm_rbf.fit(X_train_iris, y_train_iris)
svm_linear.fit(X_train_iris, y_train_iris)

# Predictions
y_pred_rbf = svm_rbf.predict(X_test_iris)
y_pred_linear = svm_linear.predict(X_test_iris)
# Accuracy and Confusion Matrix
acc_rbf = accuracy_score(y_test_iris, y_pred_rbf)
acc_linear = accuracy_score(y_test_iris, y_pred_linear)
cm_rbf = confusion_matrix(y_test_iris, y_pred_rbf)
cm_linear = confusion_matrix(y_test_iris, y_pred_linear)

```



```

# Load dataset
letter_df = pd.read_csv("/content/letter-recognition.csv") # Update path if needed
letter_df["letter"] = LabelEncoder().fit_transform(letter_df["letter"])

# Split features and labels
X = letter_df.drop("letter", axis=1)
y = letter_df["letter"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

```

```

# Train SVM
svm = SVC(kernel='rbf', probability=True)
svm.fit(X_train, y_train)
y_pred = svm.predict(X_test)
y_prob = svm.predict_proba(X_test)

# Accuracy and Confusion Matrix
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

# ROC and AUC (one-vs-rest)
y_test_bin = label_binarize(y_test, classes=np.unique(y))
n_classes = y_test_bin.shape[1]

fpr = dict()
tpr = dict()
roc_auc = dict()

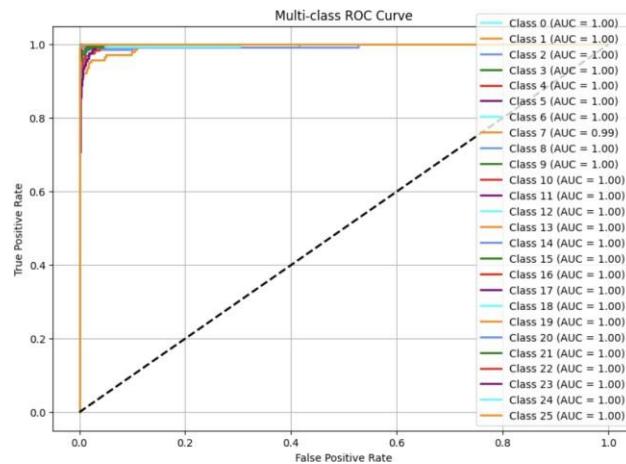
for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_test_bin[:, i], y_prob[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# Plot ROC Curve
plt.figure(figsize=(10, 7))
colors = cycle(['aqua', 'darkorange', 'cornflowerblue', 'green', 'red', 'purple'])

for i, color in zip(range(n_classes), colors):
    plt.plot(fpr[i], tpr[i], color=color, lw=2,
             label=f'Class {i} (AUC = {roc_auc[i]:0.2f})')

plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Multi-class ROC Curve")
plt.legend(loc="lower right")
plt.grid()
plt.show()

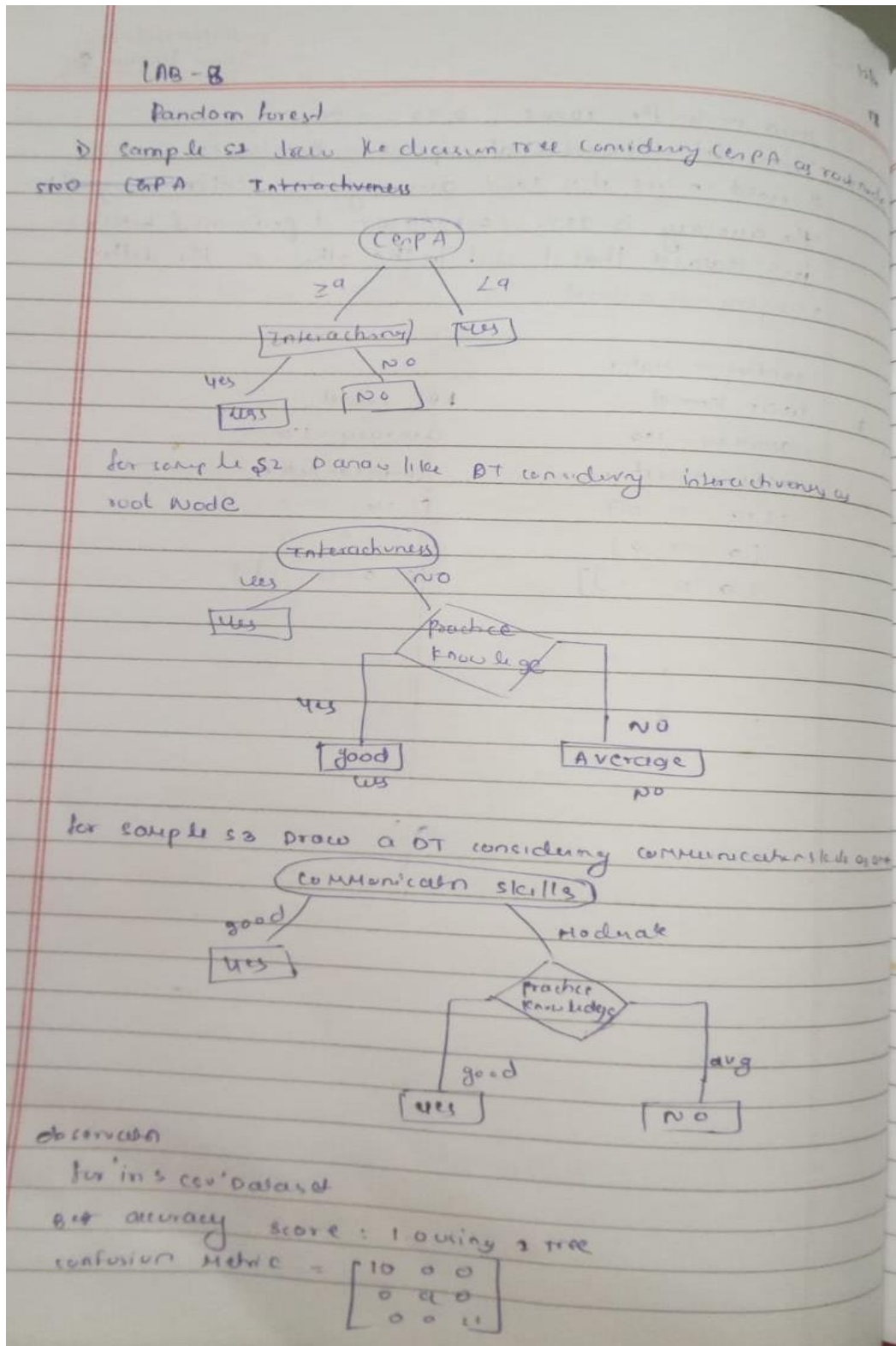
```



Program 8

8) Implement Random forest ensemble method on a given dataset

Screenshots:



Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
import matplotlib.pyplot as plt

# Load the dataset
df = pd.read_csv("iris.csv") # Adjust filename if needed

# Prepare data
X = df.drop(columns=["species"]) # Assuming 'species' is the target column
y = df["species"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Default Random Forest with 10 trees
rf_default = RandomForestClassifier(n_estimators=10, random_state=42)
rf_default.fit(X_train, y_train)
y_pred_default = rf_default.predict(X_test)
acc_default = accuracy_score(y_test, y_pred_default)
conf_matrix_default = confusion_matrix(y_test, y_pred_default)

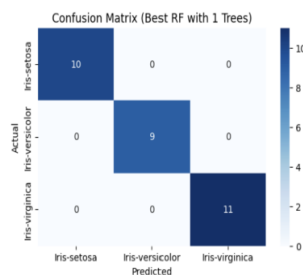
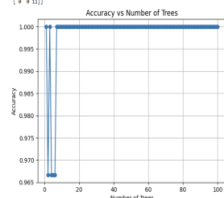
print(f"Default RF (10 trees) Accuracy: {acc_default}")
print("Confusion Matrix:\n", conf_matrix_default)

# Try different numbers of trees to find the best
best_acc = 0
best_n = 10
acc_list = []

for n in range(1, 101):
    rf = RandomForestClassifier(n_estimators=n, random_state=42)
    rf.fit(X_train, y_train)
    y_pred = rf.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    acc_list.append((n, acc))
    if acc > best_acc:
        best_acc = acc
        best_n = n
        best_conf_matrix = confusion_matrix(y_test, y_pred)

print(f"\nBest Accuracy: {best_acc} using {best_n} trees")
print("Best Confusion Matrix:\n", best_conf_matrix)
# Plot accuracy vs number of trees
x_vals, y_vals = zip(*acc_list)
plt.plot(x_vals, y_vals, marker='o')
plt.title("Accuracy vs Number of Trees")
plt.xlabel("Number of Trees")
plt.ylabel("Accuracy")
plt.grid(True)
plt.show()
```

Default RF (10 trees) accuracy: 1.0
Confusion Matrix:
[[9 0 0]
 [0 9 0]
 [0 0 11]]



Program 9

9) Implement Boosting ensemble method on a given dataset.

Screenshots:

Bafna Gold
 Date: 12/11 Page: 19

LAB -9

Implementation Boosting ensemble method on A
Given dataset

considering adaboost Alg show that decision stump
calculations steps for the attribute CERP A

step 1	CERP A	Intelligence	PK	CS	Job Profile
	≥ 9	Yes	good	Good	Yes
	< 9	No	good	Moderate	Yes
	≥ 9	No	poor Avg	Moderate	No
	< 9	No	poor Avg	Good	No
	≥ 9	Yes	go	Mod	Yes
	≥ 9	Yes	go	Moderate	Yes

step 2 = $\sum_{i=1}^n H_i(d_j) \cdot w_i(d_j)$

$H_i(d_j)$ = correct prediction

$w_i(d_j)$ = wrong prediction

$E_{cgpa} = 2 \times \frac{1}{6} = 0.333$

CERP A	PJO	ATof	W
≥ 9	Y	Y	$\frac{1}{6}$
< 9	N	Y	$\frac{1}{6}$
≥ 9	Y	N	$\frac{1}{6}$
< 9	N	N	$\frac{1}{6}$
≥ 9	Y	Y	$\frac{1}{6}$
≥ 9	Y	Y	$\frac{1}{6}$

step 3 = normalize weights

compute weight for each weak classifier

$\alpha_{cgpa} = \frac{1}{2} \ln \frac{(1 - E_{cgpa})}{E_{cgpa}}$

$\frac{1}{2} \ln \frac{(1 - 0.333)}{0.333}$

$\alpha_{cgpa} = 0.347$

step 4 = $Z_{CERP A} = w_1(\text{correct classifier instance}) + \dots + w_n(\text{correct classifier instance}) + e^{-\alpha_{cgpa}}$

$\frac{1}{6} \times 4 \times e^{-0.347} + \frac{1}{6} \times 2 \times e^{+0.347}$

$Z_{CERP A} = 0.9028$

2025.05.19 1

$$wt(d_j)_{i+1} = \frac{wt(d_j)_{CPA} \text{ of correct instance } e^{-u(d_j, CPA)}}{2 CPA}$$

$$wt(d_j)_{i+1} = \frac{\frac{1}{6} \times e^{-0.347}}{0.9428} = 0.1249 \quad \text{--- correct}$$

$$wt(d_1)_{i+1} = \frac{\frac{1}{6} \times e^{+0.347}}{0.9428} = 0.2501$$

The table

CPA	Predicted job offer	Actual job offer	Weight
≥ 9	Yes	Yes	0.1249
< 9	NO	Yes	0.2501
≥ 9	Yes	NO	0.2501
< 9	NO	NO	0.1249 0.1249
≥ 9	Yes	Yes	0.1249
≥ 9	Yes	Yes	0.1249

```

Code: import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay

# Load dataset
data = pd.read_csv('income.csv')

# Display basic info
print("First five rows:")
print(data.head())
print(f"\nDataset shape: {data.shape}")

# Define features and target
target_column = 'income_level'
y = data[target_column]
X = data.drop(columns=[target_column])

# Identify categorical vs numerical columns
categorical_cols = X.select_dtypes(include=['object', 'category']).columns.tolist()
numerical_cols = X.select_dtypes(include=['int64', 'float64']).columns.tolist()
print(f"\nNumerical columns: {numerical_cols}")
print(f"Categorical columns: {categorical_cols}")

# Preprocessor: scale numericals, one-hot encode categorical
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_cols)
    ]
)

# Initial AdaBoost model with 10 estimators
pipeline = Pipeline([
    ('preprocess', preprocessor),
    ('clf', AdaBoostClassifier(n_estimators=10, random_state=42))
])

# Split into train/test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Train and evaluate initial model
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
initial_acc = accuracy_score(y_test, y_pred)
print(f"Initial test accuracy (n_estimators=10): {initial_acc:.4f}")

# Hyperparameter tuning: find best n_estimators
tree_counts = list(range(10, 201, 10)) # 10,20,...,200
cv_scores = []
for n in tree_counts:
    model = Pipeline([

```

```

        ('preprocess', preprocessor),
        ('clf', AdaBoostClassifier(n_estimators=n, random_state=42))
    ])
    scores = cross_val_score(
        model, X_train, y_train, cv=5, scoring='accuracy', n_jobs=-1
    )
    mean_score = scores.mean()
    cv_scores.append(mean_score)
    print(f"n_estimators={n}: CV mean accuracy={mean_score:.4f}")

# Plot CV accuracy vs. number of estimators
plt.figure()
plt.plot(tree_counts, cv_scores, marker='o')
plt.title('AdaBoost CV Accuracy vs. n_estimators')
plt.xlabel('Number of Estimators')
plt.ylabel('CV Mean Accuracy')
plt.grid(True)
plt.tight_layout()
plt.show()

# Determine optimal number of trees
best_score = max(cv_scores)
best_n = tree_counts[cv_scores.index(best_score)]
print(f"\nBest CV accuracy={best_score:.4f} with n_estimators={best_n}")

# Retrain and evaluate best model
best_model = Pipeline([
    ('preprocess', preprocessor),
    ('clf', AdaBoostClassifier(n_estimators=best_n, random_state=42))
])
best_model.fit(X_train, y_train)
y_best = best_model.predict(X_test)
best_test_acc = accuracy_score(y_test, y_best)
print(f"Test accuracy with best n_estimators ({best_n}): {best_test_acc:.4f}")

# Plot comparison of initial vs. best test accuracy
plt.figure()
plt.bar(['n=10', f'n={best_n}'], [initial_acc, best_test_acc])
plt.title('Test Accuracy: Initial vs. Optimized')
plt.ylabel('Accuracy')
plt.ylim(0, 1)
plt.tight_layout()
plt.show()

# Plot confusion matrix for best model
cm = confusion_matrix(y_test, y_best)
labels = best_model.named_steps['clf'].classes_
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
plt.figure()
disp.plot(cmap=plt.cm.Blues)
plt.title('Confusion Matrix for Best AdaBoost Model')
plt.tight_layout()
plt.show()

```

Dataset shape: (48842, 7)

Numerical columns: ['age', 'fnlwgt', 'education_num', 'capital_gain', 'capital_loss', 'hours_per_week']

Categorical columns: []

Initial test accuracy (n_estimators=10): 0.8257

n_estimators=10: CV mean accuracy=0.8201

n_estimators=20: CV mean accuracy=0.8228

n_estimators=30: CV mean accuracy=0.8250

n_estimators=40: CV mean accuracy=0.8291

n_estimators=50: CV mean accuracy=0.8291

n_estimators=60: CV mean accuracy=0.8305

n_estimators=70: CV mean accuracy=0.8309

n_estimators=80: CV mean accuracy=0.8316

n_estimators=90: CV mean accuracy=0.8316

n_estimators=100: CV mean accuracy=0.8320

n_estimators=110: CV mean accuracy=0.8321

n_estimators=120: CV mean accuracy=0.8323

n_estimators=130: CV mean accuracy=0.8322

n_estimators=140: CV mean accuracy=0.8327

n_estimators=150: CV mean accuracy=0.8327

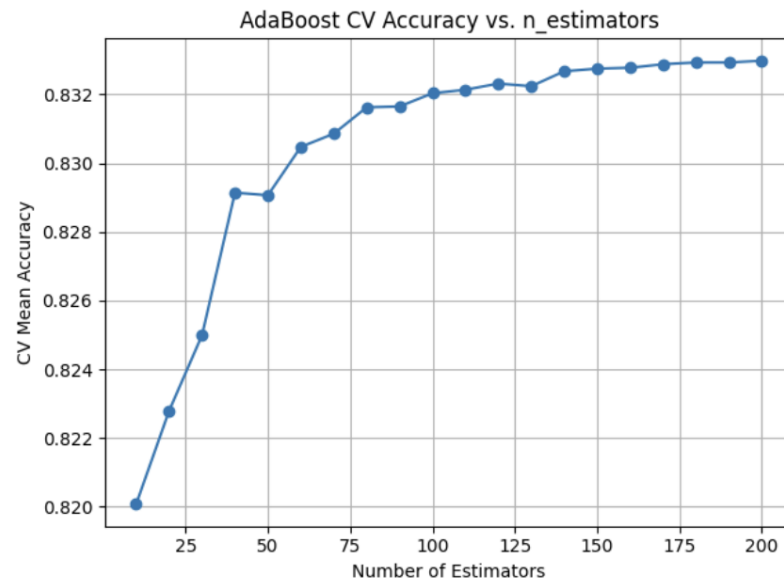
n_estimators=160: CV mean accuracy=0.8328

n_estimators=170: CV mean accuracy=0.8329

n_estimators=180: CV mean accuracy=0.8329

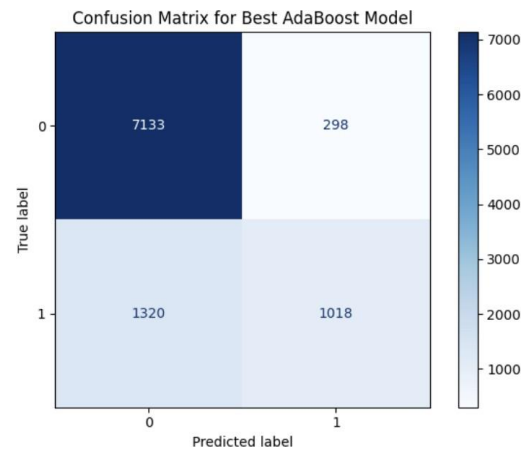
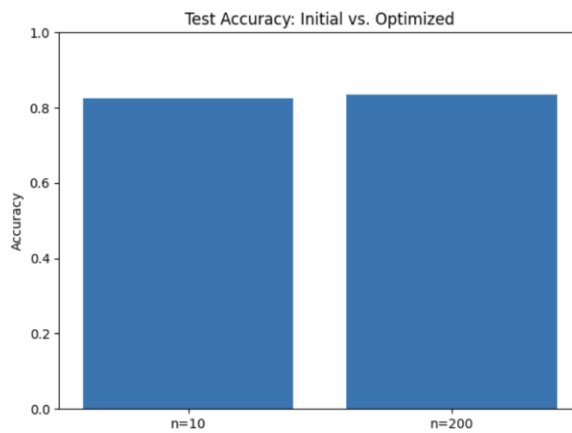
n_estimators=190: CV mean accuracy=0.8329

n_estimators=200: CV mean accuracy=0.8330



Best CV accuracy=0.8330 with n_estimators=200

Test accuracy with best n_estimators (200): 0.8344



Program 10

10) Build k-Means algorithm to cluster a set of data stored in a .CSV file.

Screenshots:

LAB-10

Bafna Gold
Date: n/a Page: 21

Build k-means Alg to cluster a set of data stored in a csv file

→ compute 2 cluster using k-means alg for clustering. here
initial cluster centers are (1.0, 1.0) and (5.0, 7.0)

execute for 3 iterations

record no	A	B
P ₁	1.0	1.0
P ₂	1.5	2.0
P ₃	3.0	4.0
P ₄	5.0	7.0
P ₅	3.5	5.0
P ₆	4.5	5.0
P ₇	3.5	4.5

record no	C ₁ (1.0, 1.0)	C ₂ (5.0, 7.0)	cluster
P ₁ (1.0, 1.0)	0	7.21	1
P ₂ (1.5, 2.0)	1.12	6.12	1
P ₃ (3.0, 4.0)	3.61	3.61	1
P ₄ (5.0, 7.0)	7.21	0.0	2
P ₅ (3.5, 5.0)	4.12	2.6	2
P ₆ (4.5, 5.0)	5.31	2.06	2
P ₇ (3.5, 4.5)	4.30	2.92	2

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$$\sqrt{(1.0 - 1.5)^2 + (1.0 - 2.0)^2} =$$

$$C_1 = (1.0 + 1.5 + 3.0) / 3, (1.0 + 2.0 + 4.0) / 3$$

$$C_1 = (1.83, 2.33)$$

$$C_2 = (5.0 + 3.5 + 4.5 + 3.5) / 3, (7.0 + 5.0 + 5.0 + 4.5) / 3$$

$$C_2 = (4.12, 5.37)$$

P NO.	$C_1 (1.83, 12.33)$	$(4.12, 15.35)$
$P_1 (1.0, 1.6)$	1.83	5.37
$P_2 (1.5, 2.6)$	0.47	0.27
$P_3 (3.0, 4.0)$	2.04	1.77
$P_4 (5.0, 7.0)$	5.64	1.83
$P_5 (3.5, 5.0)$	3.15	0.72
$P_6 (4.5, 5.0)$	3.78	0.53
$P_7 (3.5, 4.5)$	2.74	1.07
$C_1 = (1.0 + 1.6) / 2$	$(1.0 + 2.0) / 2$	$\hat{C}_2 = (3.0 + 5.0 + 3.5 + 4.5) / 4$
$= 1.25, 1.6$		$= 3.9$
		$(4.5 + 7.0 + 5.0) / 3 = 5.5$
Observation		
for iris-csv dataset:		
The elbow plot shows a sharp elbow at 1025 indicating 3 clusters is the optimal choice for the petal length in the 'iris' dataset		

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

def load_data(csv_path='iris.csv'):
    """
    Try loading from csv_path; if not found, load via sklearn.
    Expects columns: sepal_length, sepal_width, petal_length, petal_width, species.
    Returns DataFrame with a 'species' column.
    """
    try:
        df = pd.read_csv(csv_path)
        # Fixed typo here: use c.strip().replace, not ace()
        df.columns = [c.strip().replace(' ', '_') for c in df.columns]
    except FileNotFoundError:
        iris = load_iris()
        df = pd.DataFrame(
            data=np.c_[iris['data'], iris['target']],
            columns=iris['feature_names'] + ['target']
        )
        df.columns = [c.strip().replace(' (cm)', '').replace(' ', '_')
                      for c in df.columns]
        df['species'] = df['target'].map(lambda x: iris['target_names'][int(x)])
    return df

def preprocess(df):
    """
    Select only petal_length & petal_width, then standard-scale.
    Returns scaled numpy array.
    """
    X = df[['petal_length', 'petal_width']].values
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)
    return X_scaled, scaler

def plot_elbow(X_scaled, max_k=10):
    """
    Compute KMeans inertia for k=1..max_k and plot the elbow curve.
    Returns list of inertias.
    """
    inertias = []
    ks = range(1, max_k + 1)
    for k in ks:
        km = KMeans(n_clusters=k, random_state=42)
        km.fit(X_scaled)
        inertias.append(km.inertia_)
    plt.figure(figsize=(6, 4))
```

```

plt.plot(ks, inertias, 'o-', linewidth=2)
plt.xlabel('Number of clusters (k)')
plt.ylabel('Inertia')
plt.title('Elbow Method for Optimal k')
plt.xticks(ks)
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
return inertias

def run_kmeans(X_scaled, k):
    """
    Fit KMeans with k clusters, return labels and fitted model.
    """
    km = KMeans(n_clusters=k, random_state=42)
    labels = km.fit_predict(X_scaled)
    return km, labels

def plot_confusion(df, labels, k):
    """
    Builds and displays a confusion matrix comparing true species vs. cluster.
    """
    species_names = df['species'].unique()
    species_to_num = {name: idx for idx, name in enumerate(species_names)}
    true_nums = df['species'].map(species_to_num)

    cm = confusion_matrix(true_nums, labels)
    disp = ConfusionMatrixDisplay(
        confusion_matrix=cm,
        display_labels=[f"Cluster {i}" for i in range(k)]
    )
    fig, ax = plt.subplots(figsize=(6, 6))
    disp.plot(ax=ax, cmap='Blues', colorbar=True)
    ax.set_xlabel('Predicted Cluster')
    ax.set_ylabel('True Species')
    plt.title('K-Means Clustering Confusion Matrix')
    plt.tight_layout()
    plt.show()

    cm_df = pd.DataFrame(
        cm,
        index=[f"True: {name}" for name in species_names],
        columns=[f"Cluster {i}" for i in range(k)]
    )
    print("\nConfusion Matrix (counts):")
    print(cm_df)

def main():
    # 1) Load data
    df = load_data('iris.csv')
    if 'species' not in df.columns:
        print("Error: 'species' column not found.")
        return

    # 2) Preprocess
    X_scaled, scaler = preprocess(df)

```

```
# 3) Elbow plot to decide k
print("Generating elbow plot to find optimal k...")
inertias = plot_elbow(X_scaled, max_k=10)
```

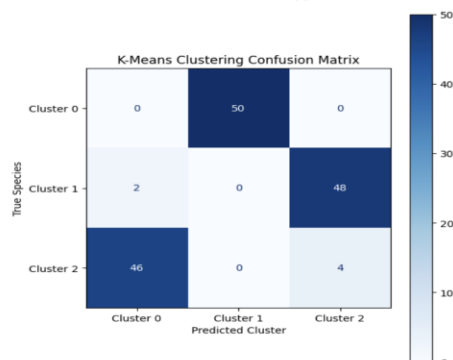
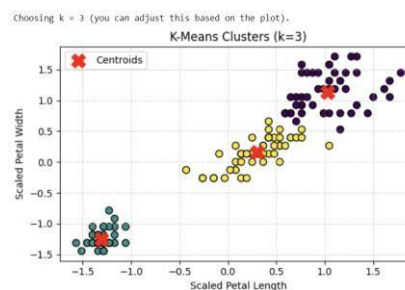
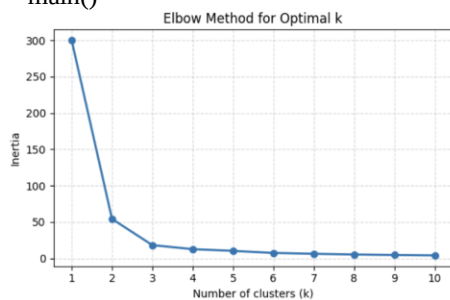
```
# 4) From the elbow you'll typically see a bend at k=3
optimal_k = 3
print(f"Choosing k = {optimal_k} (you can adjust this based on the plot).")
```

```
# 5) Run K-Means and assign clusters
km_model, labels = run_kmeans(X_scaled, optimal_k)
df['cluster'] = labels
```

```
# 6) Visualize clusters in feature space
plt.figure(figsize=(6, 4))
plt.scatter(
    X_scaled[:, 0], X_scaled[:, 1],
    c=labels, cmap='viridis', edgecolor='k', s=50
)
centroids = km_model.cluster_centers_
plt.scatter(
    centroids[:, 0], centroids[:, 1],
    marker='X', c='red', s=200, label='Centroids'
)
plt.xlabel('Scaled Petal Length')
plt.ylabel('Scaled Petal Width')
plt.title(f'K-Means Clusters (k={optimal_k})')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```

```
# 7) Confusion matrix vs. true species
plot_confusion(df, labels, optimal_k)
```

```
if __name__ == "__main__":
    main()
```



Program 11

11) Build k-Means algorithm to cluster a set of data stored in a .CSV file.

Screenshots:

Lab-11

Bafna Gold
Date: 12/14/2023

Tripled dimensionality reduction using Principle Component Analysis (PCA) Method

Reduce the dimension from 2 to 2 using PCA computer for PCA

Feature	ex1	ex2	ex3	ex4
x1	4	8	13	7
x2	11	4	5	14

eigen value

$\lambda_1 = 30.3849$

$\lambda_2 = 6.6151$

$e_1 = \begin{bmatrix} 0.5574 \\ -0.8303 \end{bmatrix}$

$e_2 = \begin{bmatrix} 0.8303 \\ 0.5574 \end{bmatrix}$

$x_1 = \frac{4+8+13+7}{4} = 8$

$x_2 = \frac{11+4+5+14}{4} = \frac{34}{4} = 8.5$

$X_{\text{centered}} = X - \text{Mean} = \begin{bmatrix} -4 & 0 & 5 & -1 \\ 2.5 & -4.5 & -3.5 & 5.5 \end{bmatrix}$

$e^T = [0.5574 \quad -0.8303]$

$z_i = e^T \cdot x_i$

$[0.5574 \quad -0.8303] \begin{bmatrix} -4 & 0 & 5 & -1 \\ 2.5 & -4.5 & -3.5 & 5.5 \end{bmatrix}$

$= \begin{bmatrix} 0.5574(-4) - 0.8303(2.5) & 0.5574(0) - 0.8303(-4.5) & 0.5574(5) - 0.8303(-3.5) & 0.5574(-1) - 0.8303(5.5) \\ (8) & (3.75) & (3.5) & (-5.125) \end{bmatrix}$

$z = \begin{bmatrix} -4.30535 \\ 3.73635 \\ 3.64305 \\ -5.12405 \end{bmatrix}$


```

Code:      import pandas as pd

           df = pd.read_csv("heart.csv")

           # Step 3: Split Features and Target
           X = df.drop("target", axis=1)
           y = df["target"]

           # Step 4: Preprocessing
           from sklearn.model_selection import train_test_split
           from sklearn.preprocessing import StandardScaler, OneHotEncoder
           from sklearn.compose import ColumnTransformer
           from sklearn.pipeline import Pipeline

           categorical_features = ["cp", "thal", "slope"]
           numerical_features = [col for col in X.columns if col not in categorical_features]

           preprocessor = ColumnTransformer(transformers=[
               ("num", StandardScaler(), numerical_features),
               ("cat", OneHotEncoder(), categorical_features)
           ])

           # Step 5: Train/Test Split
           X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

           # Step 6: Models
           from sklearn.linear_model import LogisticRegression
           from sklearn.svm import SVC
           from sklearn.ensemble import RandomForestClassifier
           from sklearn.metrics import accuracy_score

           models = {
               "Logistic Regression": LogisticRegression(max_iter=1000),
               "SVM": SVC(),
               "Random Forest": RandomForestClassifier()
           }

           # Step 7: Train and Evaluate Models (Before PCA)
           print("Accuracy Before PCA:")
           results = {}
           for name, model in models.items():
               pipeline = Pipeline(steps=[
                   ("preprocessor", preprocessor),
                   ("classifier", model)
               ])
               pipeline.fit(X_train, y_train)
               y_pred = pipeline.predict(X_test)
               acc = accuracy_score(y_test, y_pred)
               results[name] = acc
               print(f"{name}: {acc:.4f}")

           from sklearn.decomposition import PCA

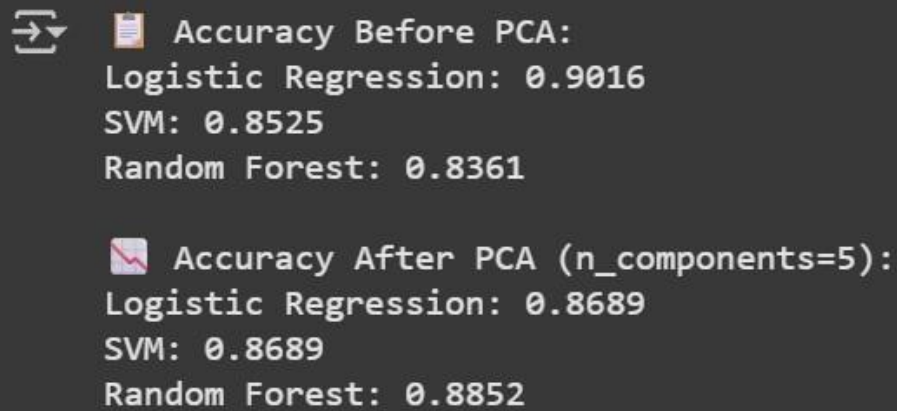
```

```

print("\nAccuracy After PCA (n_components=5):")
pca_results = {}

for name, model in models.items():
    pipeline_pca = Pipeline(steps=[
        ("preprocessor", preprocessor),
        ("pca", PCA(n_components=5)),
        ("classifier", model)
    ])
    pipeline_pca.fit(X_train, y_train)
    y_pred_pca = pipeline_pca.predict(X_test)
    acc_pca = accuracy_score(y_test, y_pred_pca)
    pca_results[name] = acc_pca
    print(f"{name}: {acc_pca:.4f}")

```



The terminal output displays two sections of accuracy results. The first section, titled 'Accuracy Before PCA:', shows results for Logistic Regression (0.9016), SVM (0.8525), and Random Forest (0.8361). The second section, titled 'Accuracy After PCA (n_components=5):', shows results for the same models after PCA: Logistic Regression (0.8689), SVM (0.8689), and Random Forest (0.8852). Each line is preceded by a small icon representing the model type.

```

➡ 📄 Accuracy Before PCA:
Logistic Regression: 0.9016
SVM: 0.8525
Random Forest: 0.8361

📈 Accuracy After PCA (n_components=5):
Logistic Regression: 0.8689
SVM: 0.8689
Random Forest: 0.8852

```